Homework #4 – Human Interface Computing

Austin Sternberg

My encryption system works by picking a random key from 1 – 100 each time the program is run. This is used to set alphabet for encryption, calling the random function to swap letters around using the key as a seed. In this example, only upper-case letters will be encrypted, so spaces, numbers, etc. are skipped. It also forces the plaintext to be converted to upper-case in the encryption/decryption functions. For the purposes of the assignment the plaintext is hardcoded, it does function with user submitted plaintext. The code is very modular.

*The easy/hard mode is mostly there for testing and making sure the program works. I know this makes it considerably easier to crack the key.*

An attacker attempting to quickly break my encryption has three real options:

- **Brute Force Attack**
  My encryption method would be very susceptible to a brute force attack. Since the key can only be between 1-100, trying every possible key to decrypt is easy and takes a fraction of a second. Absolute worst case, the attacker knows how the encryption system works and can write their own program to test all possible keys before even attempting to make a guess on my program. (see bruteforce.py)
- **Known Plaintext Attack**
  If the attacker were to know part or all of the plaintext, cracking the encryption would be easy. For example, if they guess the message might contain a name or common word, they could generate encrypted versions of that known word with each possible key and compare to the ciphertext. A match would reveal the key.
- **Dictionary Matching**
  An attacker could brute-force decrypt the ciphertext with all 100 possible keys and scan the outputs for English words using a dictionary. If one of the decrypted messages contains readable text or multiple valid words, it's likely they've found the correct key and plaintext.

Technically frequency analysis is possible, but because of the random letter shuffling, a small cipher text makes it relatively difficult to make this attack work. Frequency analysis is much better suited for traditional shift ciphers where an attacker can figure out a couple letters and then match the rest sequentially.